



چکیده

در این گزارش، چهار مورد از الگوریتم‌های تقریبی که اخیراً برای مسئله پوشش دیسک واحد ارائه شده‌اند شرح داده می‌شوند و پس از پیاده‌سازی، با استفاده از چند مجموعه نقطه دنیای واقعی، مورد ارزیابی تجربی قرار می‌گیرند. معیارهای ارزیابی، تعداد دیسک‌های در نظر گرفته شده و زمان اجرای هر الگوریتم خواهد بود. در نهایت عملکرد هر الگوریتم و بهترین الگوریتم‌ها برای هر معیار گزارش می‌شود.

۱ شرح مسئله

مجموعه P شامل n نقطه در صفحه داده می‌شود. هدف، پوشش تمام نقاط با استفاده از کمترین تعداد دیسک با شعاع واحد ($r = 1$) است. این مسئله، پوشش دیسک واحد (UDC)^۲ نامیده شده و یک مسئله ان پی سخت^۱ به حساب می‌آید [۱]. در کاربردهای مختلف مانند شبکه‌های بی‌سیم، تعیین موقعیت، برنامه‌ریزی حرکت، پردازش تصاویر و ... استفاده می‌شود.

۲ مرور الگوریتم‌ها

از سال ۱۹۹۱ تاکنون الگوریتم‌های زیادی ارائه شده‌اند که این مسئله را به صورت تقریبی با فاکتورهای تقریب و پیچیدگی‌های زمانی متفاوت، در نرم اقلیدسی حل می‌کنند. جدول ۱ تاریخچه الگوریتم‌های تقریبی ارائه شده برای این مسئله را نشان می‌دهد.

^۱NP-hard

^۲Unit Disk Cover

سال	پیچیدگی زمانی	فاکتور تقریب	مرجع
۱۹۹۱	$O(l^2 n^7)$	$2(1 + \frac{1}{l})$	[۲]
۱۹۹۱	$O(n \log S)$	8	[۲]
۱۹۹۵	$O(n^3 \log n)$	$O(1)$	[۳]
۲۰۰۱	$O(Kn)$	$3(1 + \frac{1}{l})^2$	[۴]
۲۰۰۷	$O(n(\log n \log \log n)^2)$	2.8334	[۵]
۲۰۱۴	$O(n \log n)$	25/6	[۶]
۲۰۱۷	$O(n \log n)$	4	[۷]
۲۰۱۸	$O(n \log n)$	4	[۸]
۲۰۱۸	—	$O(1.321^d)$	[۹]
۲۰۱۹	$O(n)$	7	[۱۰]

جدول ۱: خلاصه الگوریتم‌های تقریبی ارائه شده برای UDC

در ادامه، به ترتیب الگوریتم‌های ذیل بررسی می‌شوند:

- الگوریتم $BLMS$ که در سال ۲۰۱۷ ارائه شده است [۷].
- الگوریتم LL که در سال ۲۰۱۴ ارائه شده است [۶].
- الگوریتم DGT که در سال ۲۰۱۸ ارائه شده است [۹].
- الگوریتم $FastCover$ که در سال ۲۰۱۹ ارائه شده است [۱۰].

عنوان سه الگوریتم اول، برگرفته از حرف اول نام پژوهشگران مربوط به آن است.

۲۱ الگوریتم $BLMS$

مجموعه P را به عنوان مجموعه نقاط ورودی که در صفحه قرار دارند و C^* را پوشش دیسک بهینه برای آن در نظر بگیرید.

به خاطر داشته باشید که شعاع هر دیسک واحد برابر با ۱ است.

تعریف ۱-۲ در گراف تقاطع دیسک واحد^۲ که با $UDIG(P)$ نمایش داده می‌شود، نقاط موجود در مجموعه P رئوس را

تشکیل می‌دهند و برای هر جفت $p, q \in P$ یک یال وجود دارد اگر و تنها اگر $|pq| \leq 2$ باشد که $|pq|$ فاصله اقلیدسی بین

p و q را نشان می‌دهد.

مشاهده ۲-۲ برای دو نقطه $p, q \in P$ اگر $(p, q) \notin UDIG(P)$ آنگاه p و q نمی‌توانند با یک دیسک واحد پوشش

داده شوند.

^۲Unit Disk Intersection Graph

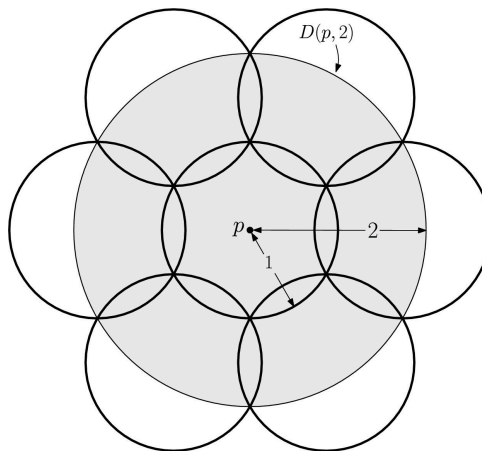
تعریف ۲-۳ یک مجموعه مستقل در $UDIG(P)$ ، زیرمجموعه‌ای مانند I از مجموعه P است، به طوری که هیچ یالی بین جفت نقطه‌های موجود در I وجود ندارد. همچنین I مجموعه مستقل حداکثری^۴ نامیده می‌شود، اگر برای هر $p \in P \setminus I$ مجموعه $I \cup \{p\}$ در $UDIG(P)$ مستقل نباشد.

فرض کنید I مجموعه مستقل حداکثری در $UDIG(P)$ باشد. طبق مشاهده ۲ اندازه هر مجموعه مستقل در $UDIG(P)$ یک حد پایین^۵ برای تعداد دیسک‌های مورد نیاز به منظور پوشش P است. بنابراین خواهیم داشت:

$$(1) \quad |I| \leq |C^*|$$

مطابق شکل ۱ برای پوشش یک دیسک با شعاع ۲، هفت دیسک واحد با شعاع ۱ لازم و کافی است. بر همین اساس، یک الگوریتم با فاکتور تقریب ۷ برای مسئله UDC به دست می‌آید.

فرض کنید I یک مجموعه مستقل حداکثری دلخواه در $UDIG(P)$ باشد. برای هر نقطه $p \in I$ فرض کنید $D(p, 2)$ یک دیسک با مرکزیت نقطه p و شعاع ۲ باشد. همچنین در نظر داشته باشید که $d(p)$ دیسک واحدی است که نقطه p را پوشش می‌دهد. علاوه بر این، تمام نقاطی که از مجموعه P توسط $d(p)$ پوشش داده می‌شوند، در $D(p, 2)$ وجود دارند. بنابراین با پوشش $D(p, 2)$ به وسیله ۷ دیسک واحد، به ازای همه $p \in I$ ، الگوریتمی با فاکتور تقریب ۷ حاصل می‌شود.



شکل ۱: پوشش $D(p, 2)$ با دیسک‌های واحد

در ادامه خواهید دید که چگونه می‌توان فاکتور تقریب را به ۴ کاهش داد. p را سمت چپ‌ترین نقطه در مجموعه P در نظر بگیرید. در موارد خاص که مقادیر x نقاط با هم برابر است، برای انتخاب سمت چپ‌ترین نقطه، کمتر بودن مقدار y را ملاک قرار

^۴Maximal Independent Set

^۵Lower Bound

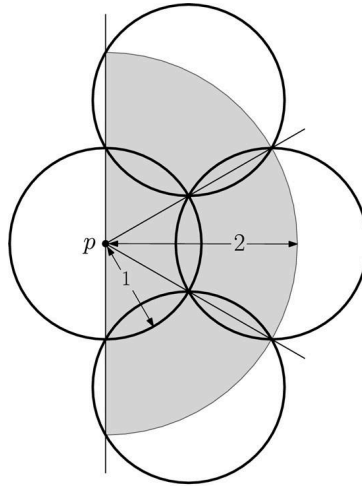
می‌دهیم. فرض کنید خط عمودی l از نقطه p عبور کند؛ $R(p)$ اشتراک $D(p, 2)$ با نیم‌صفحه^۶ سمت راست l خواهد بود؛

یعنی $R(p)$ نیم‌دیسک سمت راست $D(p, 2)$ است (مطابق شکل ۲).

همان‌طور که قبلاً هم توضیح داده شد، تمام نقاط مجموعه P که با $d(p)$ پوشش داده شده‌اند، در $D(p, 2)$ و به‌تبع آن در

$R(p)$ نیز قرار دارند. مطابق شکل ۲، نیم‌دیسک $R(p)$ می‌تواند با ۴ دیسک واحد پوشش داده شود. قسمت دوم شکل، حالتی

از موقعیت قرار گرفتن ۷ نقطه را نشان می‌دهد که برای پوشش آنها حداقل به ۴ دیسک واحد نیاز است.



شکل ۲: پوشش $R(p)$ با دیسک‌های واحد

برای نقطه‌ای مانند p و مجموعه نقاط I فاصله $d(p, I)$ برابر است با کمترین فاصله اقلیدسی بین p و هر نقطه موجود در I . اگر

مجموعه I تهی باشد، فاصله بینهایت در نظر گرفته می‌شود. الگوریتم ۱، الگوریتم پیشنهادی با فاکتور تقریب ۴ را نشان می‌دهد.

خروجی آن، مجموعه‌ای از دیسک‌های واحد با نام C است که مجموعه نقاط P را پوشش می‌دهند.

```

1:  $C = \emptyset$ 
2:  $I = \emptyset$ 
3:  $L =$  List of points in  $P$  sorted from left to right
4: while  $L$  is not empty do
5:   if  $d(p, I) > 2$  then
6:     Cover  $R(p)$  by 4 unit disks  $c_1, c_2, c_3, c_4$ 
7:      $C = C \cup \{c_1, c_2, c_3, c_4\}$ 
8:      $I = I \cup \{p\}$ 
9:   end if
10:   $L = L - \{p\}$ 
11: end while
12: return  $C$ 

```

الگوریتم ۱: نسخه اولیه $BLMS$

در هر تکرار الگوریتم ۱، نقطه p به مجموعه I اضافه می‌شود، اگر و تنها اگر $d(p, I) > 2$ باشد. بنابراین p در $UDIG(P)$

به هیچ نقطه‌ای از I متصل نیست. همچنین بعد از خاتمه الگوریتم، I یک مجموعه مستقل حداکثری خواهد بود.

^۶Half-plane

قضیه ۲-۴ فاکتور تقریب الگوریتم ۱ برای مسئله پوشش دیسک واحد، ۴ است.

برهان. مجموعه نقاط I و مجموعه دیسک‌های واحد C را در پایان الگوریتم، در نظر بگیرید. براساس رابطه ۱ می‌دانیم که نامساوی $|I| \leq |C^*|$ برقرار است. به‌ازای نقاط $p \in I$ هر نقطه $q \in P$ در یک نیم‌دیسک $R(p)$ قرار دارد (ممکن است $q = p$ باشد). چون برای هر نقطه $p \in I$ نیم‌دیسک $R(p)$ با ۴ دیسک واحد پوشش داده می‌شود، مجموعه C مجموعه P را پوشش می‌دهد. بنابراین رابطه $|C| \leq 4|I| \leq 4|C^*|$ برقرار است. ■

پیچیدگی زمانی الگوریتم ۱ برابر $O(n \log n + n \cdot t(d))$ است. $t(d)$ پیچیدگی زمانی محاسبه فاصله $d(p, I)$ را نشان می‌دهد. محاسبه این فاصله با پیچیدگی زمانی $O(\log^2 n)$ قابل انجام است [۱۱]. بنابراین پیچیدگی نهایی الگوریتم، $O(n \log^2 n)$ است که در ادامه با استفاده از تکنیک جاروی صفحه، بهبود می‌یابد.

به‌جای محاسبه فاصله $d(p, I)$ کافی است بدانیم فاصله نقطه p از مجموعه I از ۲ بیشتر است یا نه؛ یعنی به یک مسئله تصمیم‌گیری تبدیل شود. به‌تدریج که یک نقطه جدید مانند p به مجموعه I اضافه می‌شود، نقاط مجموعه P که در نیم‌دیسک $R(p)$ قرار دارند، حذف می‌شوند. بر همین اساس، یک الگوریتم با استفاده از تکنیک جاروی صفحه و پیچیدگی زمانی $O(n \log n)$ ارائه می‌شود.

در الگوریتم ۲، ابتدا نقاط بر اساس مؤلفه x از چپ به راست مرتب شده و در صف رخدادها درج می‌شوند. خط جارو به‌صورت عمودی از چپ به راست بر روی نقاط حرکت می‌کند. به هر نقطه که می‌رسد، اگر نقطه انتهایی یک نیم‌دیسک باشد، نقطه ابتدایی مربوط به آن نیم‌دیسک را از درخت وضعیت حذف می‌کند. در غیر این صورت، دو نقطه همسایه از بالا و دو نقطه همسایه از پایین را در درخت وضعیت برای این نقطه پیدا می‌کند. اگر فاصله آن با حداقل یکی از این ۴ همسایه کمتر یا مساوی ۲ باشد، نقطه مورد نظر قبلاً در یک نیم‌دیسک قرار گرفته و تحت پوشش است. در غیر این صورت، یک نیم‌دیسک به شعاع ۲ و مرکز آن نقطه در نظر گرفته می‌شود و مرکز ۴ دیسک واحد پوشش‌دهنده آن نیم‌دیسک محاسبه و به‌عنوان جواب گزارش می‌شود.

```

1: Initialize an empty event queue  $Q$ . Insert the points in ascending order of their  $x$ -coordinates into  $Q$ .
2: Initialize an empty BST status structure  $T$ .
3: Initialize an empty list  $C$ .
4: while  $Q$  is not empty do
5:   Determine the next event point  $p$  in  $Q$  and delete it.
6:   if  $p$  is an end-point then
7:     Delete the start-point of the corresponding half-disk from  $T$ .
8:   else
9:     Find the 2 top and the 2 bottom neighbors of  $p$  in  $T$ .
10:    if The distance between  $p$  and all of these 4 neighbors is greater than 2 then
11:      Calculate center points of the 4 unit disks which cover half-disk of point  $p$  and insert them into  $C$ .
12:    Insert  $p$  into  $T$ .
13:    Insert the end-point  $q = (p_x + 2, p_y)$  into  $Q$ .
14:  end if
15:  end if
16: end while
17: return  $C$ 

```

الگوریتم ۲: نسخه بهبود یافته $BLMS$ با تکنیک جاروی صفحه

۲۲ الگوریتم LL

در این الگوریتم، صفحه به نوارهای عمودی با عرض $\sqrt{3}$ تقسیم می‌شود. از هر نوار، یک جواب تقریبی با مرتب‌سازی نقاط براساس مؤلفه y به صورت نزولی به دست می‌آید. نقطه بعدی درون یک نوار که هنوز پوشش داده نشده است، با قرار دادن یک دیسک در پایین‌ترین مکان ممکن، پوشش داده می‌شود. مرکز این دیسک‌ها، روی خطوط عمودی که نوارها را به دو قسمت تقسیم می‌کنند قرار می‌گیرد. جواب نهایی با اجتماع جواب همه نوارها حاصل می‌شود. این سامانه نواری، ۵ مرتبه به سمت راست و هر دفعه به اندازه $\frac{\sqrt{3}}{6}$ شیفت داده می‌شود. در هر شیفت، یک جواب به دست می‌آید. از بین این ۶ جواب، آن جوابی که کمترین دیسک را استفاده کرده باشد به عنوان جواب نهایی در نظر گرفته می‌شود. جزئیات بیشتر در الگوریتم ۳ موجود است. فاکتور تقریب این الگوریتم $4.17 \approx 25/6$ است.

```

1: Disk-Centers  $\leftarrow \emptyset$ , min  $\leftarrow n + 1$ ;
2: Sort  $P$  w.r.t  $x$ -coordinate in  $O(n \log n)$  time;
3: for  $i \in \{0, 1, 2, 3, 4, 5\}$  do
4:   current  $\leftarrow 1$ ,  $C \leftarrow \emptyset$ , right  $\leftarrow P[1]_x + (i\sqrt{3})/6$ ;
5:   while current  $\leq n$  do
6:     index  $\leftarrow$  current;
7:     while  $P[\text{current}]_x < \text{right}$  and current  $\leq n$  do
8:       current  $\leftarrow$  current + 1;
9:     end while
10:     $x\text{-of-restriction-line} \leftarrow \text{right} - \sqrt{3}/2$ , segments  $\leftarrow \emptyset$ ;
11:    for  $j \leftarrow \text{index}$  to current - 1 do
12:       $d \leftarrow P[j]_x - x\text{-of-restriction-line}$ ,  $y \leftarrow \sqrt{1 - d^2}$ ;
13:      Create a segment  $s$  and insert it into segments;
14:    end for
15:    Sort segments in non-ascending order and greedily stab them; put stabbing points in  $C$ ;
16:    Increment right by a multiple of  $\sqrt{3}$  s.t.  $P[\text{current}] - \text{right} \leq \sqrt{3}$ ;
17:  end while
18:  if  $|C| < \text{min}$  then
19:    Disk-Centers  $\leftarrow C$ , min  $\leftarrow |C|$ ;
20:  end if
21: end for
22: return Disk-Centers;

```

الگوریتم ۳: محاسبه موقعیت دیسک‌های واحد با استفاده از LL

۲۳ الگوریتم DGT

این الگوریتم، ساده و برخط است. به ازای هر نقطه‌ای که تاکنون تحت پوشش قرار نگرفته است، یک دیسک واحد به مرکز آن نقطه ایجاد می‌شود. فاکتور تقریب آن در صفحه، ۵ است. در فضای با ابعاد d دارای فاکتور تقریب $O(1.321^d)$ است. برای مشاهده توصیف سطح بالا، به الگوریتم ۴ مراجعه نمایید.

```
1: Disk-Centers  $\leftarrow \emptyset$ ;  
2: for  $p \in P$  do  
3:   if the distance from  $p$  to the nearest point in Disk-Centers is  $> 1$  then  
4:     Disk-Centers  $\leftarrow$  Disk-Centers  $\cup \{p\}$ ;  
5:   end if  
6: end for  
7: return Disk-Centers;
```

الگوریتم ۴: محاسبه موقعیت دیسک‌های واحد با استفاده از DGT

۲۴ الگوریتم FastCover

در این الگوریتم، از یک شبکه^۶ با مربع‌های به ضلع $\sqrt{2}$ استفاده می‌شود. هر مربع از این شبکه، می‌تواند توسط یک دیسک به شعاع واحد محاط شود. به ازای هر نقطه، اگر توسط یکی از دیسک‌هایی که قبلاً قرار گرفته است تحت پوشش باشد، عملی انجام نمی‌شود؛ در غیر این صورت، یک دیسک واحد به مرکز مربعی که آن نقطه درونش قرار گرفته است، ایجاد می‌شود. در پیاده‌سازی برای جستجوی سریع‌تر، از یک جدول درهم‌ساز^۷ استفاده می‌شود. در این جدول، مختصات مرکز دیسک‌هایی که اضافه شده‌اند ذخیره می‌شود. برای جلوگیری از مشکلات اعداد اعشاری، از یک جفت عدد صحیح برای نمایش مرکز هر دیسک استفاده می‌شود. مختصات حقیقی می‌تواند با ضرب کردن هر عدد صحیح در $\sqrt{2}$ و اضافه کردن $1/\sqrt{2} = \sqrt{2}/2$ به آن به دست بیاید. این فرآیند در الگوریتم ۵ قابل ملاحظه است.

این الگوریتم دارای فاکتور تقریب ۷ و پیچیدگی زمانی $O(n)$ است. یک الگوریتم برخط به حساب می‌آید و هیچ پیش‌پردازشی مثل مرتب‌سازی روی نقاط انجام نمی‌دهد. به اندازه $O(s)$ حافظه اضافی مصرف می‌کند که s نشان‌دهنده اندازه پوشش تولید شده است.

^۶Grid

^۷Hash table

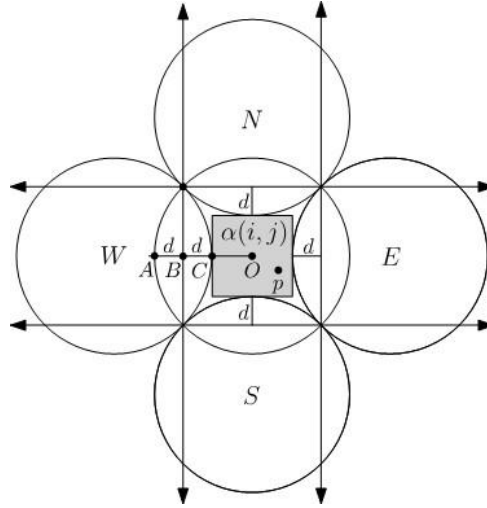
```

1:  $\mathcal{H} \leftarrow \emptyset$ ; Disk-Centers  $\leftarrow \emptyset$ ;
2: for  $p \in P$  do
3:    $i \leftarrow \lfloor p_x/\sqrt{2} \rfloor$ ;  $j \leftarrow \lfloor p_y/\sqrt{2} \rfloor$ ;
4:   if  $(i, j) \notin \mathcal{H}$  then
5:      $(\sqrt{2}i + 1/\sqrt{2},$ 
       insert  $(i, j)$  into  $\mathcal{H}$  and  $\sqrt{2}j + 1/\sqrt{2})$  into Disk-Centers;
6:   end if
7: end for
8: return Disk-Centers;

```

الگوریتم ۵: محاسبه موقعیت دیسک‌های واحد با استفاده از *FastCover*

این الگوریتم را می‌توان کمی بهبود داد. وقتی که یک نقطه در یک مربع از شبکه قرار می‌گیرد، ممکن است توسط ۴ دیسک واحد که مربوط به مربع‌های همسایه است و قبلاً اضافه شده‌اند تحت پوشش باشد (مطابق شکل ۳). بنابراین بهتر است برای کاهش تعداد دیسک‌ها این شرایط نیز بررسی شود. در الگوریتم ۶ جزئیات نسخه بهبودیافته ذکر شده است.



شکل ۳: امکان پوشش یک نقطه با دیسک‌های مربع‌های همسایه

```

1:  $\mathcal{H} \leftarrow \emptyset$ ; Disk-Centers  $\leftarrow \emptyset$ ;
2: for  $p \in P$  do
3:    $i \leftarrow \lfloor p_x/\sqrt{2} \rfloor$ ;  $j \leftarrow \lfloor p_y/\sqrt{2} \rfloor$ ;
4:   if  $(i, j) \in \mathcal{H}$  then continue
5:   else if  $p_x \geq \sqrt{2}(i + 1.5) - 1$  and  $(i + 1, j) \in \mathcal{H}$  and  $\text{dist}(p, D(i + 1, j)) \leq 1$  then
6:     continue
7:   else if  $p_x \leq \sqrt{2}(i - 0.5) + 1$  and  $(i - 1, j) \in \mathcal{H}$  and  $\text{dist}(p, D(i - 1, j)) \leq 1$  then
8:     continue
9:   else if  $p_y \geq \sqrt{2}(j + 1.5) - 1$  and  $(i, j + 1) \in \mathcal{H}$  and  $\text{dist}(p, D(i, j + 1)) \leq 1$  then
10:    continue
11:   else if  $p_y \leq \sqrt{2}(j - 0.5) + 1$  and  $(i, j - 1) \in \mathcal{H}$  and  $\text{dist}(p, D(i, j - 1)) \leq 1$  then
12:    continue
13:   else
14:     insert  $(i, j)$  into  $\mathcal{H}$  and  $(\sqrt{2}i + 1/\sqrt{2}, \sqrt{2}j + 1/\sqrt{2})$  into Disk-Centers;
15:   end if
16: end for
17: return Disk-Centers;

```

الگوریتم ۶: محاسبه موقعیت دیسک‌های واحد با استفاده از *FastCover* +

بهبود دیگری می‌توان روی الگوریتم اعمال نمود. اگر نقاط موجود در دو دیسک مجاور را بتوان با یک دیسک پوشش داد، یعنی قطر مجموعه نقاط هر دو دیسک کمتر از ۲ باشد، می‌توان آن دو دیسک را ادغام نمود. برای ادغام، یک دیسک واحد به مرکز وسط قطر مجموعه نقاط در نظر گرفته می‌شود. برای اینکه محاسبه قطر، تأثیری روی پیچیدگی زمانی الگوریتم نداشته باشد، به همراه هر دیسک واحدی که ایجاد می‌شود، یک مستطیل مرزی نیز برای مجموعه نقاط موجود در آن نگهداری می‌شود. جزئیات بیشتر در الگوریتم ۷ ذکر شده است.

```

1:  $\mathcal{H} \leftarrow \emptyset$ ; Disk-Centers  $\leftarrow \emptyset$ ;
2: for  $p \in P$  do
3:    $i \leftarrow \lfloor p_x / \sqrt{2} \rfloor$ ;  $j \leftarrow \lfloor p_y / \sqrt{2} \rfloor$ ;
4:   if  $(i, j) \in \mathcal{H}$  then update  $B(i, j)$  using  $p$ ;  $\triangleright p$  is already covered by  $D(i, j)$ 
5:   else if east/west/north/south neighbor covers  $p$  then update its bounding box;
6:   else insert  $(i, j)$  into  $\mathcal{H}$  and initialize  $B(i, j)$  using  $p$ ;
7:   end if
8: end for
9: while there is a grid-disk  $(i, j) \in \mathcal{H}$  not yet considered do
10:   if  $\exists$  grid-disk  $(k, l) \in \mathcal{H}$  s.t.  $|i - k| \leq 1, |j - l| \leq 1$  and  $\text{diagonal}(B(i, j) \cup B(k, l)) \leq 2$  then
11:     remove  $(i, j)$  and  $(k, l)$  from  $\mathcal{H}$ ; add center of merged box to Disk-Centers;
12:   end if
13: end while
14: for every grid-disk  $(i, j) \in \mathcal{H}$  do
15:    $(\sqrt{2}i + 1/\sqrt{2},$ 
       insert  $\sqrt{2}j + 1/\sqrt{2})$  into Disk-Centers;
16: end for
17: return Disk-Centers;

```

الگوریتم ۷: محاسبه موقعیت دیسک‌های واحد با استفاده از *FastCover++*

۳ ارزیابی

همه الگوریتم‌ها با زبان $C++$ و کتابخانه *CGAL* پیاده‌سازی شده‌اند. هر کدام از آنها بر روی ۱۰ مجموعه نقطه دنیای واقعی اجرا شده‌اند. جدول ۲ نتایج ارزیابی را نشان می‌دهد. هر الگوریتم، ۵ بار بر روی هر مجموعه نقطه اجرا شده است. هر خانه کمترین تعداد دیسک و کمترین زمان پردازش بر حسب ثانیه از بین این ۵ اجرا را نشان می‌دهد. منظور از $LL-1P$ ، اجرای یک مرحله‌ای الگوریتم ۳ به جای شش مرحله است.

	LL	LL-1P	BLMS	DGT	FastCover	FastCover+	FastCover++
birch۳	۹۹۹۸۹, ۰.۱۸	۹۹۹۹۱, ۰.۰۳	۹۹۹۹۴, ۰.۰۵	۹۹۹۹۳, ۰.۰۷	۹۹۹۹۶, ۰.۰۲	۹۹۹۹۵, ۰.۰۲	۹۹۹۸۰, ۰.۰۸
monalisa	۱۰۰۰۰۰, ۰.۱۵	۱۰۰۰۰۰, ۰.۰۳	۱۰۰۰۰۰, ۰.۱۱	۱۰۰۰۰۰, ۰.۰۸	۱۰۰۰۰۰, ۰.۰۲	۱۰۰۰۰۰, ۰.۰۲	۱۰۰۰۰۰, ۰.۰۸
usa	۱۱۵۴۷۵, ۰.۱۷	۱۱۵۴۷۵, ۰.۰۴	۱۱۵۴۷۵, ۰.۱۲	۱۱۵۴۷۵, ۰.۰۹	۱۱۵۴۷۵, ۰.۰۲	۱۱۵۴۷۵, ۰.۰۳	۱۱۵۴۷۵, ۰.۱۰
KDDCU۲D	۱۱۴۷, ۰.۱۹	۱۱۵۲, ۰.۰۴	۱۶۹۲, ۰.۱۰	۱۶۲۶, ۰.۰۱	۱۴۱۸, ۰.۰۱	۱۳۷۴, ۰.۰۱	۱۲۵۷, ۰.۰۱
europe	۱۶۸۲۵۳, ۰.۳۵	۱۶۸۲۷۱, ۰.۰۶	۱۶۸۰۸۸, ۰.۱۹	۱۶۸۰۶۹, ۰.۱۶	۱۶۸۳۳۳, ۰.۰۳	۱۶۸۲۷۷, ۰.۰۴	۱۶۷۸۱۱, ۰.۲۰
wildfires	۶۲۲, ۳.۷۴	۶۲۲, ۰.۸۸	۸۴۲, ۱.۲۱	۷۸۷, ۰.۱۲	۶۶۳, ۰.۰۴	۶۳۷, ۰.۰۴	۶۳۰, ۰.۰۶
world	۶۶۶۷, ۲.۸۴	۶۶۸۰, ۰.۵۱	۹۱۴۵, ۰.۷۹	۱۰۹۸۰, ۰.۱۵	۷۸۷۴, ۰.۰۳	۷۵۷۶, ۰.۰۴	۶۹۶۷, ۰.۰۷
nyctaxi	۲۵, ۱۳.۸۴	۲۶, ۳.۰۹	۳۲, ۲.۹۰	۳۱, ۰.۱۳	۳۴, ۰.۰۵	۳۱, ۰.۰۵	۲۵, ۰.۱۰
uber	۳, ۲۱.۰۶	۳, ۴.۷۵	۵, ۴.۰۳	۵, ۰.۱۹	۵, ۰.۰۶	۴, ۰.۰۶	۴, ۰.۱۶
hail۲۰۱۵	۸۸۸, ۳۹.۸۳	۸۸۹, ۹.۸۲	۱۱۹۳, ۱۱.۱۹	۱۱۲۸, ۰.۷۴	۹۰۱, ۰.۲۸	۸۶۰, ۰.۲۸	۸۴۷, ۰.۴۲

جدول ۲: نتایج ارزیابی الگوریتم‌ها

۴ نتیجه‌گیری

در مجموعه نقطه‌های دنیای واقعی، اگر معیار با اهمیت‌تر برای ارزیابی، تعداد دیسک‌های استفاده شده باشد، الگوریتم LL عملکرد بهتری دارد. هرچند که الگوریتم $FastCover^{++}$ در برخی موارد، هم از لحاظ تعداد دیسک‌ها و هم زمان اجرا، عملکرد بهتری داشته است.

اگر معیار مهم‌تر، زمان اجرا باشد الگوریتم $FastCover$ عملکرد بهتری داشته است؛ اما چون تفاوت چندانی با نسخه بهبودیافته خود که تعداد دیسک‌های کمتری تولید می‌کند ندارد، استفاده از $FastCover^{++}$ پیشنهاد می‌شود.

اگر هردو معیار تعداد دیسک‌ها و زمان اجرا با اهمیت باشد، استفاده از الگوریتم $FastCover^{++}$ توصیه می‌شود؛ چرا که تعادل خوبی بین هردو معیار برقرار می‌کند [۱۲].

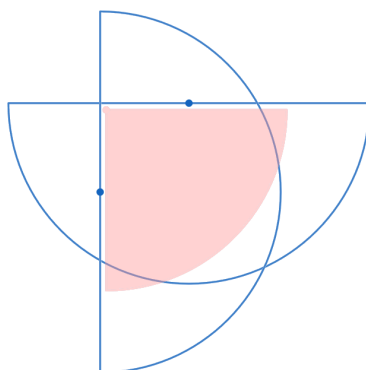
- [١] R. J. Fowler, M. S. Paterson, and S. L. Tanimoto, "Optimal packing and covering in the plane are NP-complete," *Information processing letters*, vol. ١٢, no. ٣, pp. ١٣٣–١٣٧, ١٩٨١.
- [٢] T. F. Gonzalez, "Covering a set of points in multidimensional space," *Information processing letters*, vol. ٤٠, no. ٤, pp. ١٨١–١٨٨, ١٩٩١.
- [٣] H. Brönnimann and M. T. Goodrich, "Almost optimal set covers in finite VC-dimension," *Discrete & Computational Geometry*, vol. ١٤, no. ٤, pp. ٤٦٣–٤٧٩, ١٩٩٥.
- [٤] M. Franceschetti, M. Cook, and J. Bruck, "A geometric theorem for approximate disk covering algorithms," ٢٠٠١.
- [٥] B. Fu, Z. Chen, and M. Abdelguerfi, "An almost linear time 2.8334 -approximation algorithm for the disc covering problem," in *International Conference on Algorithmic Applications in Management*, ٢٠٠٧, pp. ٣١٧–٣٢٤.
- [٦] P. Liu and D. Lu, "A fast $2.5/6$ -approximation for the minimum unit disk cover problem," *arXiv*, ٢٠١٤.
- [٧] A. Biniaz, P. Liu, A. Maheshwari, and M. Smid, "Approximation algorithms for the unit disk cover problem in $2D$ and $3D$," *Computational Geometry*, vol. ٦٠, pp. ٨–١٨, ٢٠١٧.
- [٨] M. Imanparast and S. N. Hashemi, "A simple greedy approximation algorithm for the unit disk cover problem," *AUT Journal of Mathematics and Computing*, vol. ١, no. ١, pp. ٤٧–٥٥, ٢٠٢٠.
- [٩] A. Dumitrescu, A. Ghosh, and C. D. Tóth, "Online unit covering in Euclidean space," *Theoretical Computer Science*, vol. ٨٠٩, pp. ٢١٨–٢٣٠, ٢٠٢٠.
- [١٠] A. Ghosh, B. Hicks, and R. Shevchenko, "Unit disk cover for massive point sets," in *International Symposium on Experimental Algorithms*, ٢٠١٩, pp. ١٤٢–١٥٧.

- [11] J. L. Bentley and J. B. Saxe, “Decomposable searching problems I. Static-to-dynamic transformation,” *Journal of Algorithms*, vol. 1, no. 4, pp. 301–358, 1980.
- [12] R. Friederich, M. Graham, A. Ghosh, B. Hicks, and R. Shevchenko, “Experiments with Unit Disk Cover Algorithms for Covering Massive Pointsets,” *arXiv*, 2022.

پیوست

همان‌طور که در جدول ۱ ملاحظه شد، بهترین الگوریتم‌هایی که تاکنون برای مسئله UDC ارائه شده‌اند، دارای فاکتور تقریب ۴ هستند. اینجانب، توجه زیادی به این مسئله با هدف بهبود فاکتور تقریب و نگارش مقاله نمودم. ایده‌های مختلفی برای بهبود فاکتور تقریب به ذهنم رسید که پس از بررسی‌های فراوان متوجه شدم برخی از آنها اشتباه است و برخی دیگر را به‌دلیل کمبود زمان نتوانستم به‌طور دقیق بررسی کنم. در ادامه تعدادی از آنها را بیان می‌کنم.

ایده اول، با هدف کاهش فاکتور تقریب از ۴ به ۳ بود. در شکل ۲ نشان داده شد که برای پوشش یک نیم‌دیسک به شعاع ۲، دقیقاً به ۴ دیسک واحد احتیاج است. بر اساس این شکل، برای پوشش یک ربع‌دیسک نیز دقیقاً به ۳ دیسک واحد احتیاج است. با فرض اینکه اشتراک دو نیم‌دیسک به شعاع ۲ حداکثر با یک ربع‌دیسک قابل پوشش است، می‌توان یک‌بار مطابق الگوریتم ۲ خط جارو را از چپ به راست و بار دیگر از بالا به پایین حرکت داد. پس از بررسی مشخص شد که این ایده عملی نیست. چون فرض اولیه اشتباه است و حالت‌هایی وجود دارد که اشتراک دو نیم‌دیسک بیشتر از حد تصور می‌شود و نمی‌توان آن را با یک ربع‌دیسک پوشش داد (مطابق شکل ۴).



شکل ۴: عدم امکان پوشش اشتراک دو نیم‌دیسک با ربع‌دیسک

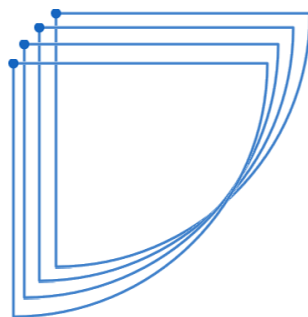
ایده دوم نیز با هدف کاهش فاکتور تقریب از ۴ به ۳ بود. به‌جای نیم‌دیسک‌های با شعاع ۲، ربع‌دیسک‌هایی به شعاع ۲ در نظر گرفته می‌شود که هرکدام با ۳ دیسک واحد قابل پوشش هستند. در الگوریتم ۸ جزئیات ایده بیان شده است. در این الگوریتم، خط جارو از بالا به پایین بر روی نقاط حرکت می‌کند. پس از بررسی، مشخص شد که این ایده نیز، عملی نیست. زیرا مانند شکل ۵ حالت‌هایی وجود دارد که ربع‌دیسک‌های زیادی با یکدیگر همپوشانی پیدا می‌کنند و در نهایت فاکتور تقریب، $O(n)$ می‌شود.

```

1: Initialize an empty event queue  $Q$ . Insert the points in descending order of their  $y$ -coordinates into  $Q$ . If
   two points have the same  $y$ -coordinate, the one with smaller  $x$ -coordinate has higher priority.
2: Initialize an empty BST status structure  $T$ .
3: Initialize an empty list  $C$ .
4: while  $Q$  is not empty do
5:   Determine the next event point  $p$  in  $Q$  and delete it.
6:   if  $p$  is an end-point then
7:     Delete the start-point of the corresponding quarter disk from  $T$ .
8:   else
9:     Find the two left neighbors  $p', p''$  of  $p$  in  $T$ .
10:    if  $|pp'| > 2$  and  $|pp''| > 2$  then
11:      Calculate center points of the 3 unit disks which cover quarter disk of point  $p$  and insert them into
       $C$ .
12:    Insert  $p$  into  $T$ .
13:    Insert the end-point  $q = (p_x, p_y + 2)$  into  $Q$ .
14:    end if
15:  end if
16: end while
17: return  $C$ 

```

الگوریتم ۸: ایده دوم برای کاهش فاکتور تقریب به ۳



شکل ۵: حالتی که ربع‌دیسک‌های زیادی همپوشانی پیدا می‌کنند